YZT101

Karar Yapıları

if • elif • else • bileşik koşullar • iç içe kararlar • temel girdi doğrulama

Bu haftanın temel sorusu

“Bir program hangi durumda hangi yolu seçer?”

4. Hafta • 2 saat teori + 2 saat uygulama

Bu hafta ne öğreneceğiz?

Dersin sonunda yalnızca if yazmak değil, karar mantığı kurmak hedefimiz.

- `if`, `elif` ve `else` bloklarının çalışma mantığını açıklamak
- Tekli, ikili ve çok seçenekli karar yapılarını uygun problemde kullanmak
- `and`, `or`, `not` ile bileşik koşulları karar yapıları içinde kurmak
- İç içe kararların ne zaman gerekli olduğunu ayırt etmek
- Temel girdi doğrulama ile geçersiz veriyi fark etmek
- Akış diyagramındaki karar noktalarını Python koduna çevirmek

Ana fikir: Koşul yazmak ≠ karar mantığı kurmak. Bugün ikisini birlikte yapacağız.

BAŞLANGIÇ

4 ders saatlik yol haritası

Her ders saati bir öncekinin üzerine inşa edilecek.

1. Ders Saati	Karar mantığı • if • girinti • akış diyagramından koda
2. Ders Saati	if–else • sınır değerler • kod izleme • hata avı
3. Ders Saati	elif • bileşik koşullar • koşul sırası • temel doğrulama
4. Ders Saati	İç içe kararlar • bütünleşik mini uygulama • test ve hata ayıklama

Ders boyunca “tahmin et → çalıştır → karşılaştır → düzelt” döngüsünü kullanacağız.

HATIRLAMA

Önceki haftadan: Koşul nedir?

Karar yapıları Boolean ifadeleri tüketir.

- Karşılaştırma sonucu: `True` veya `False`
- `puan >= 50` → bir Boolean ifade
- `yas >= 18 and izin_var` → bileşik Boolean ifade
- `not kayitli` → Boolean değeri tersine çevirir
- Bugün bu sonuçları “hangi kod çalışacak?” sorusuna bağlayacağız

if’in koşulu önce değerlendirilir; kod bloğuna girilip girilmeyeceğine sonra karar verilir.

01

Tek yollu karar: if

Bir koşul doğru olduğunda belirli kodları çalıştırmak.

Hedefimiz: if satırını, girintili kod bloğunu ve kararın yürütme akışına etkisini net biçimde okuyabilmek.

Karar yapısı neyi değiştirir?

Program artık her satırı koşulsuz çalıştırmaz.

- Sıralı program: Satır 1 → Satır 2 → Satır 3 → ...
- Kararlı program: Bir noktada koşul değerlendirilir
- Koşul `True` ise ilgili blok çalışır
- Koşul `False` ise blok atlanır
- Sonrasında program ortak akıştan devam eder

Karar yapısı “programı bitir” değil, “hangi satırlar çalışsın?” sorusuna cevap verir.

if yapısının anatomisi

Koşul, iki nokta ve girintili blok birlikte bir bütündür.

```
puan = 72

if puan >= 50:
    print("Başarı koşulu sağlandı.")

print("Program devam ediyor.")
```

Python’da blokları süslü parantez değil, girinti belirler.

Girinti neden kritik?

Girinti yalnızca “düzenli kod” meselesi değildir; Python sözdiziminin parçasıdır.

Doğru blok

```
if sicaklik > 30:
    print("Sıcaklık yüksek")
    print("Su içmeyi unutma")

print("Ölçüm tamamlandı")
```

Yanlış / farklı anlam

```
if sicaklik > 30:
    print("Sıcaklık yüksek")

print("Su içmeyi unutma")
print("Ölçüm tamamlandı")
```

İkinci örnekte “Su içmeyi unutma” mesajı koşuldan bağımsız hale geldi.

Örnek: Eşik tabanlı uyarı

Bir değerin belirli bir sınırı geçmesi durumunda işlem yapmak.

```
bakiye = -125

if bakiye < 0:
    print("Uyarı: Bakiyeniz eksiye düştü.")

print(f"Güncel bakiye: {bakiye} TL")
```

Sınır değerini mutlaka test et: -1, 0, 1 gibi.

Kod izleme: Program hangi satırları çalıştırır?

Bir if yapısını okumak, satırların yürütme sırasını takip etmektir.

```
yas = 20

print("A")
if yas >= 18:
    print("B")
print("C")
```

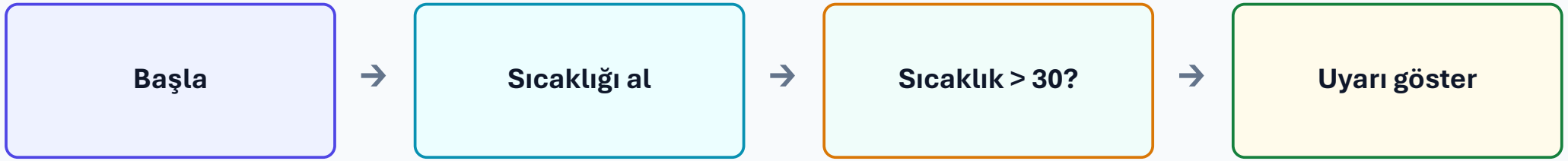
yas = 20 için

1. print("A") → çalışır
2. yas >= 18 → True
3. print("B") → çalışır
4. print("C") → çalışır

Çıktı:
A
B
C

Akış diyagramından Python koduna

Karar elması → Boolean koşul → girintili işlem bloğu



Python karşılığı

```
sicaklik = float(input("Sıcaklık: "))

if sicaklik > 30:
    print("Sıcaklık yüksek.")

print("Kontrol tamamlandı.")
```

İlk `if` hataları

Hataların bir bölümü sözdizimi, bir bölümü mantık hatasıdır.

- `if puan >= 50` → satır sonunda `:` eksik
- `if puan = 50:` → karşılaştırma yerine atama operatörü kullanılmış
- Girintinin unutulması → blok oluşmaz / hata alınır
- Yanlış karşılaştırma: `>` yerine >=
- Koşuldan bağımsız satırı yanlışlıkla blok içine almak
- Sınır değerleri test etmeden “çalışıyor” varsaymak

Sözdizimi hatası → Python durur. Mantık hatası → Python çalışır ama siz yanlış sonuç alırsınız.

Uygulama 1: Tekli karar yazalım

Önce algoritmayı cümleyle kurun, sonra koda geçin.

- Kullanıcıdan bir ürün stok miktarı alın.
- Stok 5'in altındaysa: "Stok kritik seviyede." mesajı gösterin.
- Her durumda güncel stok miktarını ekrana yazdırın.
- Test değerleri: `4`, `5`, `6`
- Soru: `stok <= 5` yazarsak davranış nasıl değişir?

Önce beklenen sonucu söyle, sonra çalıştır. Böylece test gerçekten test olur.

Kontrol noktası

Bu noktada aşağıdaki sorulara rahat cevap verebilmeliyiz.

- `if` koşulu hangi tür sonucu üretmelidir?
- Koşul `False` olduğunda girintili blok ne olur?
- Girintiden çıkan satır koşula bağlı mıdır?
- `>` ile >= arasındaki fark sınır değerlerde nasıl görünür?
- Akış diyagramındaki karar elması Python’da hangi yapıya dönüşür?

Bir sonraki adım: “Koşul yanlışsa ne yapacağız?” sorusuna `else` ile cevap vereceğiz.

02

İki yollu karar: if–else

Koşul doğruysa bir yol, yanlışsa diğer yol.

Hedefimiz: iki olasılıktan yalnızca birinin çalıştığını görmek ve sınırları hatasız tanımlamak.

İki olasılık birbirini dışlıyorsa

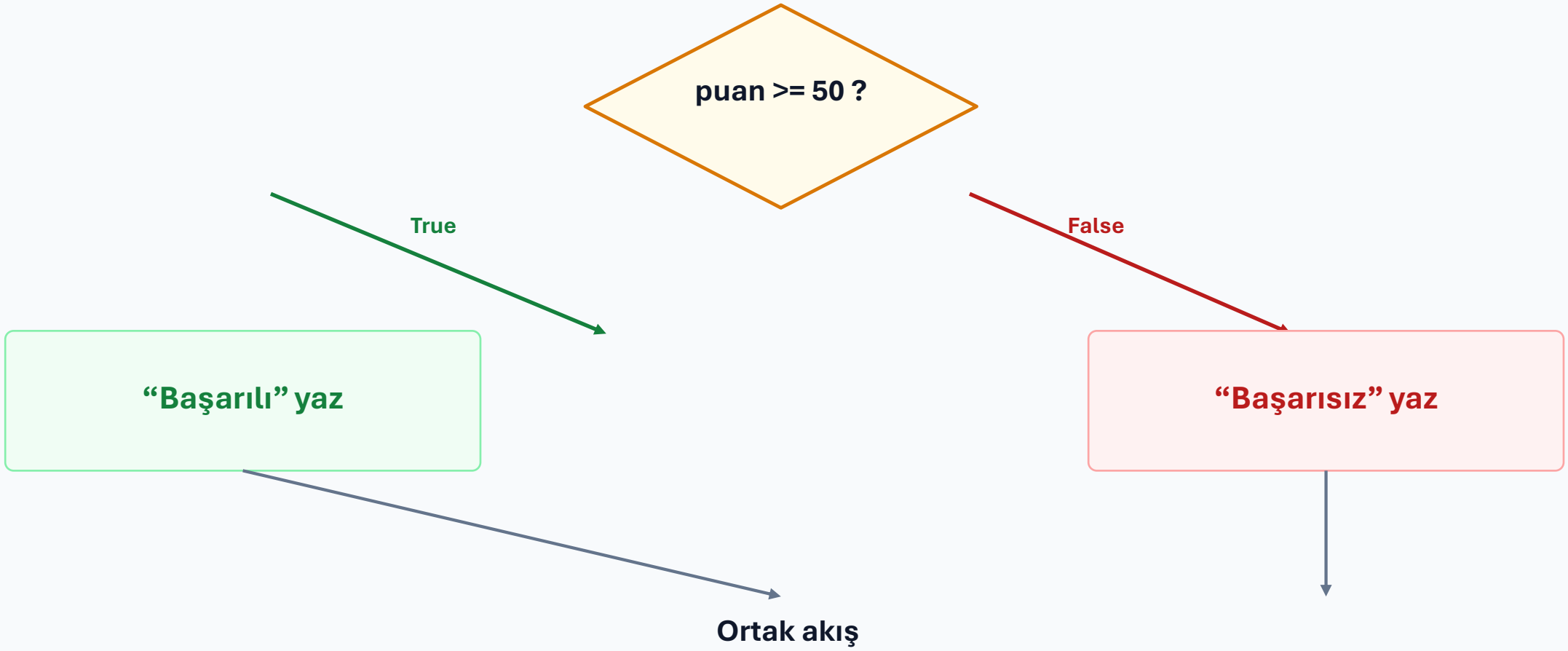
Bir çalıştırmada iki bloktan yalnızca biri yürütülür.

- `if` → koşul `True` olduğunda çalışır
- `else` → aynı koşul `False` olduğunda çalışır
- `else` kendi başına yeni bir koşul yazmaz
- İki yolun sonunda program ortak akışta birleşir
- Kullanım: geçti/kaldı, yeterli/yetersiz, tek/çift, izin/red

if ve else alternatif iki yoldur; aynı çalıştırmada ikisi birden çalışmaz.

İki yollu kararın akış diyagramı

Tek soru, iki alternatif yol, sonra yeniden birleşen akış.



Örnek: Geçti / kaldı

Sınır değeri 50 olan iki yollu sınıflandırma.

```
puan = int(input("Sınav puanı: "))

if puan >= 50:
    print("Başarılı")
else:
    print("Başarısız")

print("Değerlendirme tamamlandı.")
```

Test et: 49 → Başarısız, 50 → Başarılı, 51 → Başarılı.

Örnek: Tek mi, çift mi?

Geçen haftaki `%` operatörü karar yapısının koşuluna dönüşüyor.

```
sayi = int(input("Bir tam sayı: "))

if sayi % 2 == 0:
    print("Sayı çifttir.")
else:
    print("Sayı tektir.")
```

Eski konular kaybolmuyor; operatörler artık kararların içine giriyor.

Sınır değer düşüncesi

Karar hatalarının büyük bölümü tam sınırda ortaya çıkar.

- Kural: “18 yaş ve üzeri” → `yas >= 18`
- Kural: “18 yaşından büyük” → `yas > 18`
- Kural: “100 TL’den az” → `tutar < 100`
- Kural: “100 TL veya daha az” → `tutar <= 100`
- Her kural için: sınırın altı • sınır • sınırın üstü

Test üçlüsü: sınır − 1, sınır, sınır + 1.

`else` hangi `if`e aittir?

Girinti seviyesi, eşleşmeyi gösterir.

```
yas = 20
izin_var = False

if yas >= 18:
    if izin_var:
        print("Giriş onaylandı.")
    else:
        print("İzin belgesi eksik.")
```

Aynı girinti seviyesi, aynı yapısal seviyeyi gösterir.

Tahmin et → çalıştır → karşılaştır

Kodu çalıştırmadan önce çıktıyı zihinsel olarak yürütün.

```
sayi = 10

if sayi > 10:
    print("A")
else:
    print("B")

print("C")
```

Sorular

1. 10 > 10 sonucu nedir?
2. Hangi blok çalışır?
3. `C` her durumda yazılır mı?
4. `sayi = 11` olursa çıktı ne olur?
5. Koşulu >= yaparsak 10 için ne değişir?

Hata avı: Neler yanlış olabilir?

Çalışan kod her zaman doğru kod değildir.

- `if puan >= 50:` ama iş kuralı aslında “60 ve üzeri” olabilir
- `else` yanlış girintide olabilir
- İki blokta aynı mesaj yanlışlıkla bulunabilir
- Ortak olması gereken satır yalnızca bir bloğa konmuş olabilir
- Test sadece tek değerle yapılmış olabilir
- Kullanıcı girdisi beklenen aralıkta olmayabilir

Python “niyetinizi” değil, yazdığınız koşulu uygular.

Uygulama 2: Ücretsiz kargo kararı

İki yollu karar ve sınır değer testi.

- Kullanıcıdan sepet tutarını alın.
- Tutar `500 TL veya üzeriyse` “Kargo ücretsiz.” yazdırın.
- Aksi halde “Kargo ücreti uygulanacak.” yazdırın.
- Her durumda sepet tutarını gösterin.
- Test: `499`, `500`, `501`

Soru: “500’den fazla” denseydi hangi operatör değişirdi?

Kontrol noktası: if mi, if–else mi?

Yapıyı problem belirler.

- Yalnızca belirli durumda ek işlem varsa → tekli `if`
- İki olasılığın ikisinde de farklı işlem varsa → `if–else`
- `else` yeni koşul yazmaz; if'in False durumudur
- Ortak işlemleri blokların dışına taşımak kod tekrarını azaltır
- Sınır değerleri iki yolun doğru ayrıldığını gösterir

Bir sonraki bölümde iki seçenekten daha fazlasına geçiyoruz: `elif`.

03

Çok seçenekli karar: elif

Bir değeri üç veya daha fazla kategoriye ayırmak.

Hedefimiz: koşulların sırasını doğru kurmak, bileşik koşulları kullanmak ve geçersiz girdiyi ayırt etmek.

`elif` ne işe yarar?

İlk doğru koşul seçilir; diğerleri artık kontrol edilmez.

```
puan = 78

if puan >= 85:
    print("Çok iyi")
elif puan >= 70:
    print("İyi")
elif puan >= 50:
    print("Yeterli")
else:
    print("Yetersiz")
```

elif zincirinde “ilk True kazanır”.

Bağımsız `if`ler ile `elif` aynı şey değildir

Birinde birden fazla blok çalışabilir; diğerinde yalnızca ilk eşleşme çalışır.

Bağımsız if’ler

```
puan = 90

if puan >= 50:
    print("Geçti")
if puan >= 70:
    print("İyi")
if puan >= 85:
    print("Çok iyi")
```

if–elif zinciri

```
puan = 90

if puan >= 85:
    print("Çok iyi")
elif puan >= 70:
    print("İyi")
elif puan >= 50:
    print("Geçti")
```

90 için solda üç mesaj, sağda yalnızca “Çok iyi” çıkar.

Koşul sırası sonucu değiştirebilir

Özellikle kapsayan eşiklerde genel koşulu üste yazmak sorun yaratır.

- Yanlış sıra: önce `puan >= 50`, sonra `puan >= 85`
- 90 puan ilk koşulda yakalanır → alt koşula hiç ulaşılmaz
- Eşik tabanlı sınıflandırmada genellikle yüksek sınırdan başlanır
- Her `elif` önceki koşulların False olduğunu varsayarak çalışır
- Sıra, algoritmanın bir parçasıdır; yalnızca kod biçimi değildir

elif zinciri: önce özel/yüksek eşikler, sonra daha genel/düşük eşikler.

Örnek: Harf notu sınıflandırması

Sınırların ve sıranın birlikte tasarlanması.

```
puan = int(input("Puan (0-100): "))

if puan >= 85:
    harf = "AA"
elif puan >= 70:
    harf = "BB"
elif puan >= 50:
    harf = "CC"
else:
    harf = "FF"

print(f"Harf notu: {harf}")
```

Karar sonucu bir değişkende saklanabilir; çıktı ortak akışta tek kez üretilebilir.

Aralıklarda boşluk ve çakışma

Kategori tasarlarken bütün değerlerin tam bir kez kapsandığından emin olun.

Boşluk örneği

```
if puan >= 80:
    ...
elif puan >= 60:
    ...
elif puan < 50:
    ...
```

50–59 arası?

Çakışma / kötü sıra

```
if puan >= 50:
    ...
elif puan >= 80:
    ...
```

80+ değerler ilk koşulda yakalanır.

Hedef: Geçerli her değer bir kategoriye girsin ve amaçlanmadıkça birden fazla kategoriyle çakışmasın.

`and` ile iki şartı birlikte istemek

Karar bloğuna girmek için her iki koşul da True olmalıdır.

```
yas = 22
uyelik_aktif = True

if yas >= 18 and uyelik_aktif:
    print("Etkinliğe kayıt yapılabilir.")
else:
    print("Kayıt koşulları sağlanmıyor.")
```

and = tüm şartlar birlikte gerekli.

`or` ile alternatif şartlar

Koşullardan en az birinin True olması yeterlidir.

```
ogrenci = False
personel = True

if ogrenci or personel:
    print("Kampüs indirimi uygulanır.")
else:
    print("Standart fiyat uygulanır.")
```

or = seçeneklerden en az biri yeterli.

`not` ile koşulu tersine çevirmek

Doğruyu yanlış, yanlışı doğru yapar.

```
bakim_modu = False

if not bakim_modu:
    print("Sistem kullanıcı erişimine açık.")
else:
    print("Sistem bakımda.")
```

Koşulu sesli okuyun: `if not bakim_modu` → “bakım modu değilse”.

Bileşik koşullarda okunabilirlik

Karmaşık koşullarda parantez niyeti görünür kılar.

- `and`, `or`, `not` belirli işlem önceliğine sahiptir
- Kod doğru olsa bile okuyucu koşulu yanlış yorumlayabilir
- Mantıksal grupları parantezle açıklaştırın
- Örnek: `(ogrenci or personel) and aktif`
- Bir koşul çok uzuyorsa ara Boolean değişkenleri kullanmayı düşünün

Okunabilirlik de doğruluğun bir parçasıdır; karmaşık koşulu anlaşılır yazın.

Örnek: Etkinlik uygunluğu

Aralık kontrolü + üyelik koşulu.

```
yas = int(input("Yaş: "))
uyelik_aktif = True

if 18 <= yas <= 30 and uyelik_aktif:
    print("Gençlik etkinliğine kayıt uygun.")
else:
    print("Kayıt koşulları sağlanmıyor.")
```

Test: 17, 18, 30, 31 ve üyelik False durumu.

Uygulama 3: Yaş grubu sınıflandırması

Bir değeri birden fazla kategoriye ayırın.

- 0–12 → “Çocuk”
- 13–17 → “Ergen”
- 18–64 → “Yetişkin”
- 65 ve üzeri → “İleri yaş”
- Şimdilik negatif yaş için ayrıca “Geçersiz yaş” kontrolü ekleyin
- Test: `-1`, `0`, `12`, `13`, `17`, `18`, `64`, `65`

Kategori sayısı arttıkça test sayısı da artmalıdır.

Girdi doğrulama nedir?

Kullanıcıdan gelen her değer iş kuralına uygun olmayabilir.

- Tür dönüşümü başarılı olsa bile değer mantıksal olarak geçersiz olabilir
- Örnek: puan = `-10` veya `140`
- Önce geçerlilik kontrolü, sonra asıl sınıflandırma yapılabilir
- Geçersiz veri için açık mesaj üretmek kullanıcı deneyimini iyileştirir
- Bu hafta: tek seferlik doğrulama; tekrar isteme döngüsünü 5. haftada yapacağız

Doğrulama = “veri geldi” demek değil, “veri kurala uygun mu?” diye kontrol etmektir.

Örnek: Puan aralığını doğrulamak

Geçersiz değer sınıflandırmadan önce yakalanır.

```
puan = int(input("Puan (0-100): "))

if puan < 0 or puan > 100:
    print("Hata: Puan 0-100 arasında olmalıdır.")
elif puan >= 85:
    print("AA")
elif puan >= 70:
    print("BB")
elif puan >= 50:
    print("CC")
else:
    print("FF")
```

Geçersiz durumları erken ele almak, sonraki koşulları güvenli hale getirir.

Örnek: Menü seçimini doğrulamak

Geçerli seçenekler belirli bir küme olabilir.

```
secim = int(input("1: Ekle, 2: Sil, 3: Listele → "))

if secim == 1:
    print("Ekleme işlemi seçildi.")
elif secim == 2:
    print("Silme işlemi seçildi.")
elif secim == 3:
    print("Listeleme işlemi seçildi.")
else:
    print("Geçersiz seçim.")
```

Bu hafta doğrula ve bildir; gelecek hafta doğrula, tekrar sor ve devam et.

Şimdilik neyi yapmıyoruz?

Konuları birbirine karıştırmadan, katmanlı ilerliyoruz.

- `while` ile tekrar tekrar veri istemiyoruz → 5. hafta
- `try-except` ile harf girilmesini yakalamıyoruz → 14. hafta
- Karmaşık metin doğrulamasına girmiyoruz → metinler 7. hafta
- Bugünkü odak: karar mantığı + aralık/seçenek doğrulama
- Amaç, her yeni yapıyı önce tek başına sağlam öğrenmek

İyi öğrenme stratejisi: önce yapıyı izole et, sonra başka yapılarla birleştir.

Kontrol noktası: Çok seçenekli karar

Bu bölümün temel mantığını özetleyelim.

- `elif` zinciri yukarıdan aşağıya çalışır
- İlk `True` olan dal çalışır, geri kalanı atlanır
- Koşul sırası özellikle eşiklerde sonucu değiştirir
- `and` → hepsi gerekli, `or` → en az biri yeterli, `not` → tersine çevirir
- Geçersiz girdiyi sınıflandırmadan önce ayıklamak gerekir

Son bölüm: Bazen ikinci soruyu yalnızca ilk sorunun cevabı uygunsa sormak gerekir.

04

İç içe kararlar ve bütünleşik uygulama

Bir kararın içinde ikinci bir karar gerektiğinde.

Hedefimiz: girinti seviyelerini takip etmek, gereksiz nesting’den kaçınmak ve tüm haftayı tek problemde birleştirmek.

İkinci soru ne zaman sorulmalı?

Bazı kontroller ancak bir önceki koşul sağlandığında anlamlıdır.

- 1. koşul sağlanmadan 2. koşulu kontrol etmek gereksiz olabilir
- İçteki karar yalnızca dıştaki bloğa girildiğinde değerlendirilir
- Her yeni seviye bir girinti daha ekler
- Girinti arttıkça okuma ve hata ayıklama zorlaşır
- Bu nedenle iç içe yapıyı gerekliyse kullanın

İçteki koşul, dış koşul True olmadan hiç değerlendirilmez.

Örnek: Hesap durumu + parola

İkinci kontrol yalnızca hesap aktifse yapılır.

```
hesap_aktif = True
parola_dogru = False

if hesap_aktif:
    print("Hesap aktif.")

    if parola_dogru:
        print("Giriş başarılı.")
    else:
        print("Parola hatalı.")
else:
    print("Hesap pasif.")
```

Test kombinasyonları: (aktif/doğru), (aktif/yanlış), (pasif/doğru), (pasif/yanlış).

Test matrisi: Tüm yolları görelim

İki Boolean değişken dört kombinasyon üretir.

hesap_aktif	parola_dogru	Beklenen sonuç
True	True	Giriş başarılı
True	False	Parola hatalı
False	True	Hesap pasif
False	False	Hesap pasif

Neden son iki satırın sonucu aynı?

Çünkü hesap pasifken içteki parola kontrolüne hiç girilmiyor.

Bileşik koşul mu, iç içe karar mı?

Aynı sonuca giden iki yazım bazen farklı açıklama düzeyi sunar.

Bileşik koşul

```
if hesap_aktif and parola_dogru:
    print("Giriş başarılı")
else:
    print("Giriş yapılamadı")
```

İç içe karar

```
if hesap_aktif:
    if parola_dogru:
        print("Giriş başarılı")
    else:
        print("Parola hatalı")
else:
    print("Hesap pasif")
```

İç içe yapı, başarısızlığın nedenini ayrı ayrı açıklamak istediğimizde daha anlamlı olabilir.

Örnek: Yaş + belge kontrolü

Ön koşul sağlanırsa ikinci karar devreye girer.

```
yas = int(input("Yaş: "))
belge_var = True

if yas >= 18:
    if belge_var:
        print("Başvuru alınabilir.")
    else:
        print("Gerekli belge eksik.")
else:
    print("Yaş koşulu sağlanmıyor.")
```

Karar sırası, iş sürecinin gerçek sırasını yansıtmalı.

Gereksiz iç içe yapıdan kaçının

Girinti derinliği arttıkça kodu izlemek zorlaşır.

- Aynı seviyede çözülebilen kategorileri gereksiz yere iç içe koymayın
- Tek sonuç için çok sayıda nested `if` okunabilirliği düşürür
- Bileşik koşul veya `elif` daha uygun olabilir
- Kodun mantığını cümle halinde anlatabiliyor musunuz?
- Amaç: mümkün olan en kısa kod değil, en açık karar yapısı

Nesting kullanmak beceri; gereksiz nesting'den kaçınmak daha büyük beceri.

Hangi `else` yanlış yere bağlanmış?

Girintiyi değiştirince programın anlamı değişir.

```
yas = 20
belge_var = False

if yas >= 18:
    if belge_var:
        print("Başvuru alınabilir.")
else:
    print("Belge eksik.")
```

Kod çalışıyor olabilir; fakat yanlış `else` eşleşmesi sonucu sessizce bozabilir.

Mini uygulama: Kurs kayıt uygunluğu

Haftanın tüm yapılarını tek problemde birleştireceğiz.

- Girdi: yaş, ön test puanı, kontenjan durumu
- Geçersiz yaş veya puanı önce yakala
- Yaş en az 16 olmalı
- Ön test: 80+ → ileri grup, 60–79 → temel grup, 60 altı → uygun değil
- Kontenjan yoksa kayıt yapılamaz
- Uygun grubu ve kayıt sonucunu açık mesajla göster

Önce algoritma, sonra kod. Uzun programlarda bu adım daha da önemlidir.

Adım 1: Karar sırasını tasarla

Koddan önce iş kurallarını sıraya koyuyoruz.



Her kararın “neden bu sırada?” sorusuna bir cevabı olmalı.

Adım 2: Yalancı kod

Python sözdizimine geçmeden mantığı sade cümlelerle doğrulayın.

```
YAŞ, PUAN ve KONTENJAN bilgilerini al

EĞER yaş veya puan geçersizse
    hata mesajı göster
DEĞİLSE EĞER yaş 16'dan küçükse
    yaş koşulu sağlanmıyor
DEĞİLSE
    EĞER puan 80 veya üzeriyse
        grup = "İleri"
    DEĞİLSE EĞER puan 60 veya üzeriyse
        grup = "Temel"
    DEĞİLSE
        grup uygun değil

    Uygun grup varsa kontenjanı kontrol et
```

Yalancı kod hatasını düzeltmek, çalışan koddaki mantık hatasını bulmaktan daha ucuzdur.

Adım 3: Python kodu — doğrulama ve yaş

İlk katmanda geçersiz veriyi ve yaş koşulunu ayırıyoruz.

```
yas = int(input("Yaş: "))
puan = int(input("Ön test puanı (0-100): "))
kontenjan_var = True

if yas < 0 or puan < 0 or puan > 100:
    print("Geçersiz veri.")
elif yas < 16:
    print("Yaş koşulu sağlanmıyor.")
else:
    # Puan sınıflandırması burada devam edecek.
    pass
```

Katman 1: Önce geçersiz durumları ve ön koşulu ele.

Adım 4: Python kodu — grup ve kontenjan

İkinci katmanda puan sınıflandırması ve kayıt sonucu.

```
else:
    if puan >= 80:
        grup = "İleri"
    elif puan >= 60:
        grup = "Temel"
    else:
        grup = None

    if grup is not None:
        if kontenjan_var:
            print(f"{grup} gruba kayıt uygun.")
        else:
            print("Kontenjan dolu.")
    else:
        print("Ön test puanı yetersiz.")
```

Uzun karar zincirlerinde her girinti seviyesinin anlamını cümleyle açıklayabilmelisiniz.

Adım 5: Test senaryoları

Tek bir başarılı örnek programın doğru olduğunu kanıtlamaz.

Yaş	Puan	Kontenjan	Beklenen sonuç
-1	70	Var	Geçersiz veri
18	120	Var	Geçersiz veri
15	90	Var	Yaş koşulu sağlanmıyor
16	59	Var	Puan yetersiz
16	60	Var	Temel grup / kayıt uygun
20	80	Var	İleri grup / kayıt uygun
20	85	Yok	Kontenjan dolu

Testler; geçersiz veri, sınır değer, başarılı yol ve başarısız yolları birlikte kapsamalı.

Karar yapısında hata arama sırası

Sorunu rastgele değiştirmek yerine sistematik ilerleyin.

- 1. Girdi değerlerini doğrula
- 2. Her koşulun True/False sonucunu elle hesapla
- 3. Koşul sırasını kontrol et
- 4. Girinti seviyelerini ve `else` eşleşmelerini kontrol et
- 5. Sınır operatörlerini (`>`, >=, `<`, `<=`) kontrol et
- 6. Küçük testlerle tek tek dalları çalıştır

Debugging = rastgele değişiklik değil, hipotez kurup kontrol etmektir.

Bireysel görev: Sinema bilet sistemi

Dersin son bölümünde tek başınıza geliştirin.

- Girdi: yaş, öğrenci durumu (`True/False` olarak verilebilir), bilet türü seçimi
- Negatif yaş → geçersiz veri
- 0–6 yaş → ücretsiz
- 7–17 yaş → çocuk bileti
- 18+ → yetişkin bileti
- Yetişkin ve öğrenci ise indirim mesajı göster
- Geçersiz bilet türü seçimini ayrıca yakalayın

Teslim kriteri: çalışan kod + mantıklı koşul sırası + sınır değer testleri.

KAPANIŞ

Ders çıkış soruları

Cevapları kod yazmadan, mantığı açıklayarak düşünün.

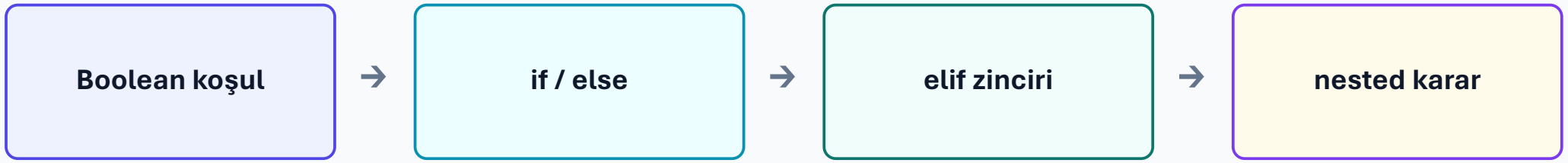
- Neden `if puan >= 50` ile `if puan > 50` aynı değildir?
- Bir `elif` zincirinde neden koşul sırası önemlidir?
- `and` ve `or` hangi iş kurallarını temsil eder?
- Bir `else`in hangi `if`e bağlı olduğunu nasıl anlarsınız?
- Girdi doğrulama neden sınıflandırmadan önce yapılmalıdır?
- İç içe karar yerine ne zaman bileşik koşul tercih edersiniz?

Programlama öğrenirken “neden?” sorusuna cevap vermek, “nasıl yazılır?” kadar önemlidir.

KAPANIŞ

Bugünün büyük resmi

Bir programın davranışını kurallara göre dallandırmayı öğrendik.



Problem → Karar noktaları → Koşullar → Bloklar → Test değerleri

Gelecek hafta: while döngüsü • sayaç • biriktirici • koşula bağlı tekrar

Karar yapıları programın “hangi yolu?”, döngüler ise “kaç kez?” sorusuna cevap verecek.